

Software platform project's doc

Introduction

This project was developed as part of the UniPD's **Software Platforms** course.

The objective is to design and implement a distributed platform that collects documents from one or more online sources, identifies documents related to a specific topic (e.g. **science and technology**), stores them, makes them searchable, and extracts useful representations for expert users.

The system follows a **service-oriented architecture**, emphasizing modularity, reproducibility, and documented design patterns.

Project Objectives

- Collect documents from external online sources;
 - Identify and filter documents related to science and technology;
 - Store documents in a persistent database;
 - Provide search and indexing capabilities;
 - Perform topic modeling to extract meaningful representations;
 - Support expert users through structured data access and analysis.
-

Project Requirements

- Use **multiple services** (microservice-oriented approach);
 - Use **Docker** for containerized deployment (recommended);
 - Services implemented primarily in **Java**;
 - Apply and document **design patterns and architectural choices**;
 - Architecture documentation is a **core evaluation component**.
-

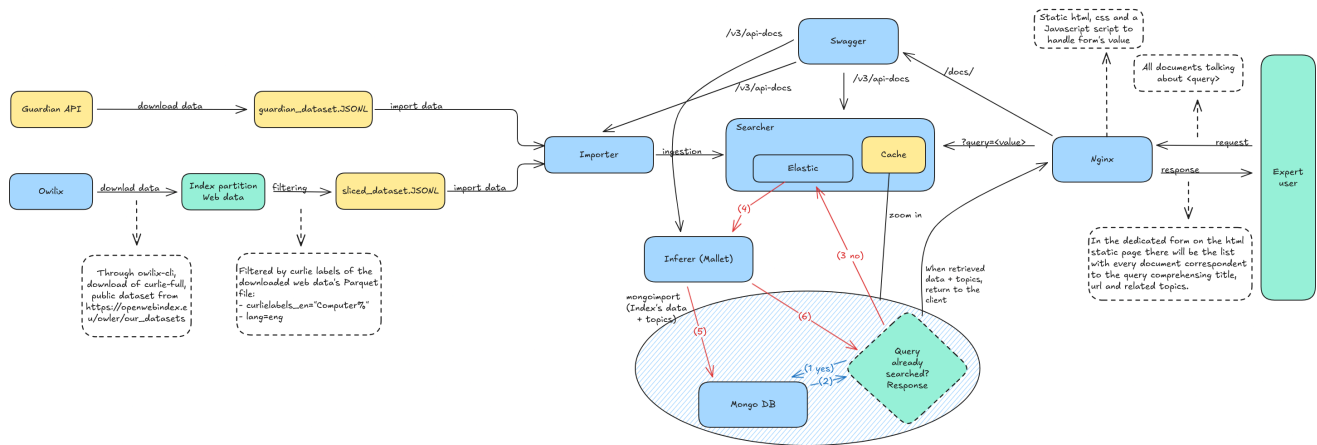
Evaluation Criteria

The project is evaluated based on:

- **Documentation**;
 - **Code quality**;
 - **Reproducibility**;
 - **Final presentation**;
 - Every group member must actively participate;
 - A slide-based presentation (e.g., PowerPoint) is recommended.
-

Project Structure

The following diagram illustrates the system pipeline:



Where the **blue figures** represent services instantiated in separate Docker containers, the **yellow ones** represent components that are useful for understanding the work pipeline but can be scripts, files or other similar components. Finally, the **green rectangles** are only logical blocks that serve the sole purpose of making the diagram more understandable even to those who are not familiar with the project and its purpose.

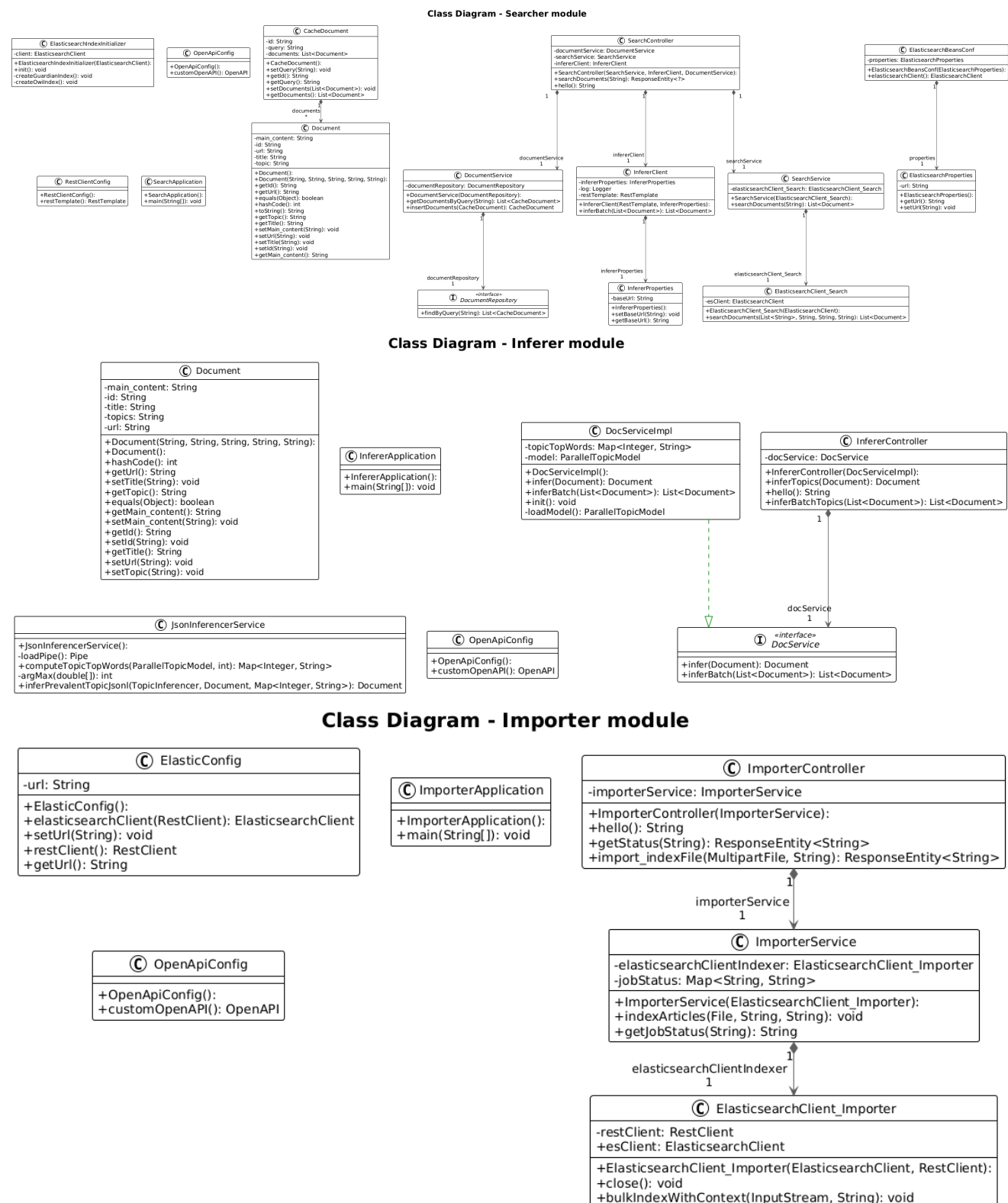
4+1 view model

Here a quick recap of what is the 4+1 architectural view model and what is its goal. The 4+1 architectural view model provides a comprehensive framework for documenting software architecture through multiple perspectives, each addressing different stakeholder concerns.

Logical View

It serves primarily **analysts** and **designers** who need to understand the system's functionality.

Class diagrams

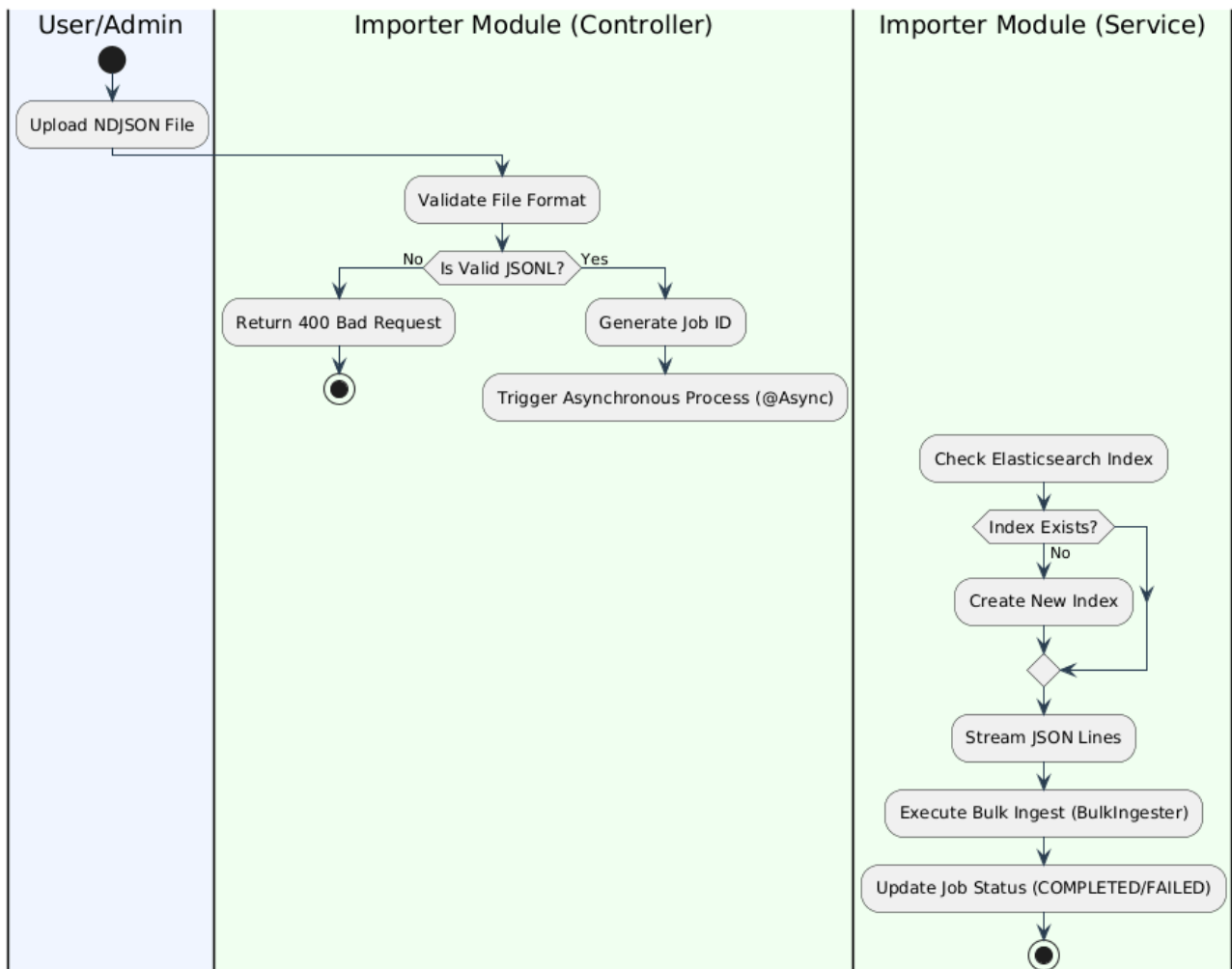


Process View

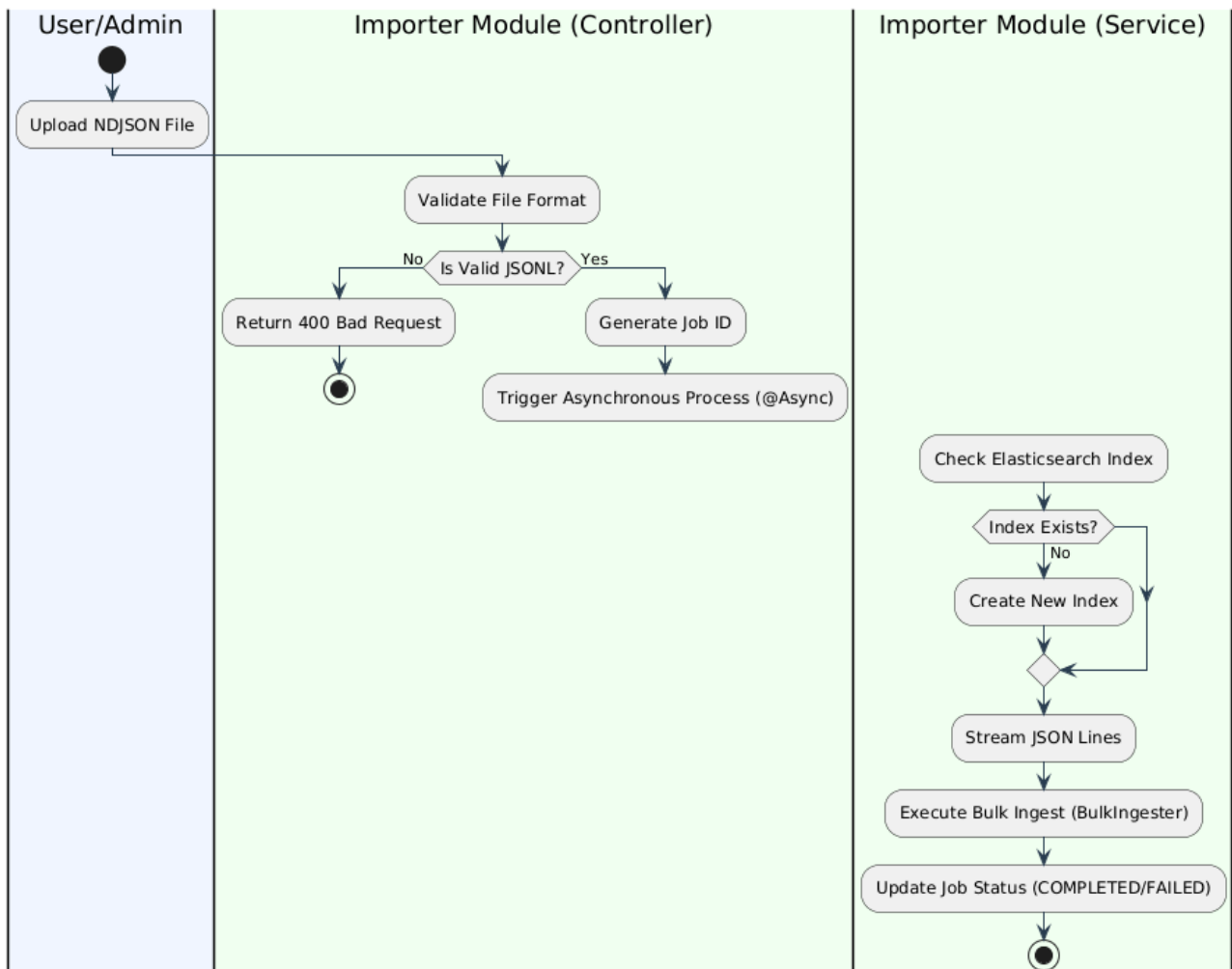
Particularly valuable for **integrators** and **engineers** concerned with system performance, scalability and throughput.

Activity diagrams

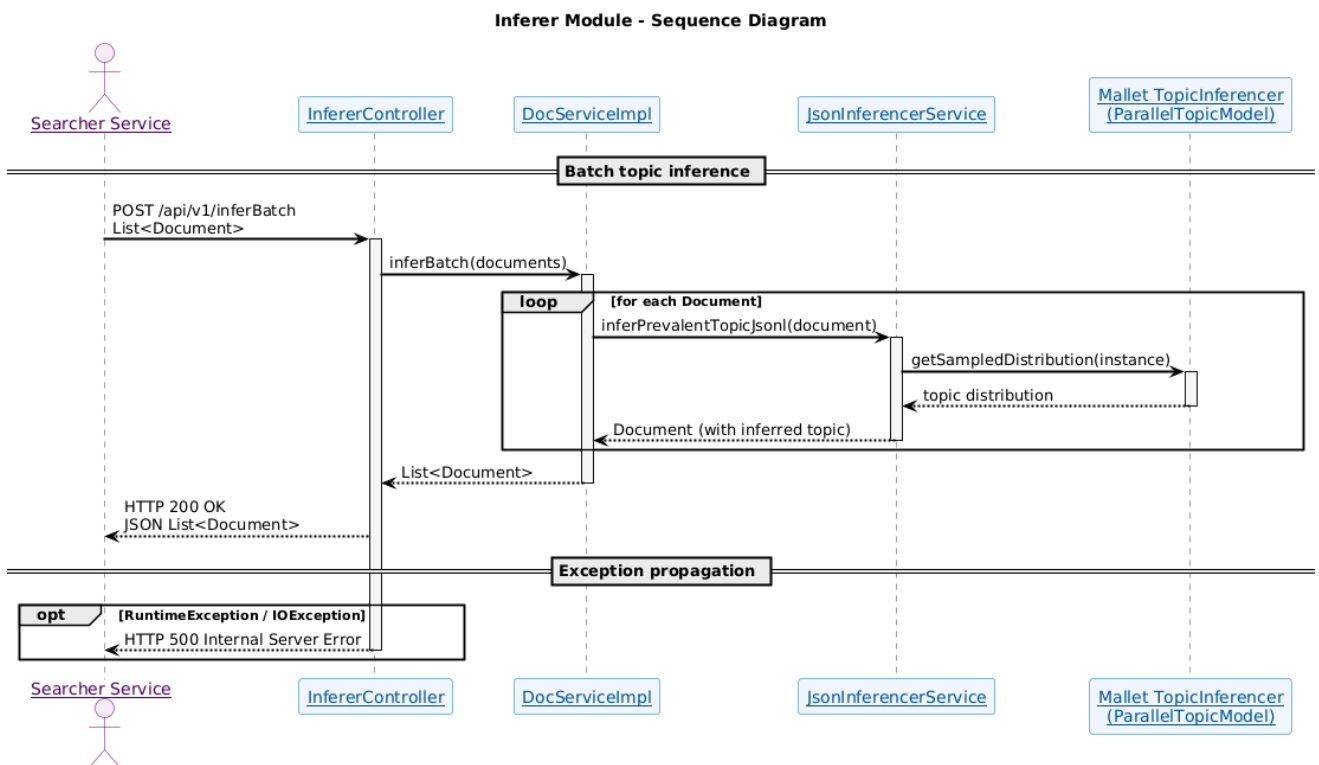
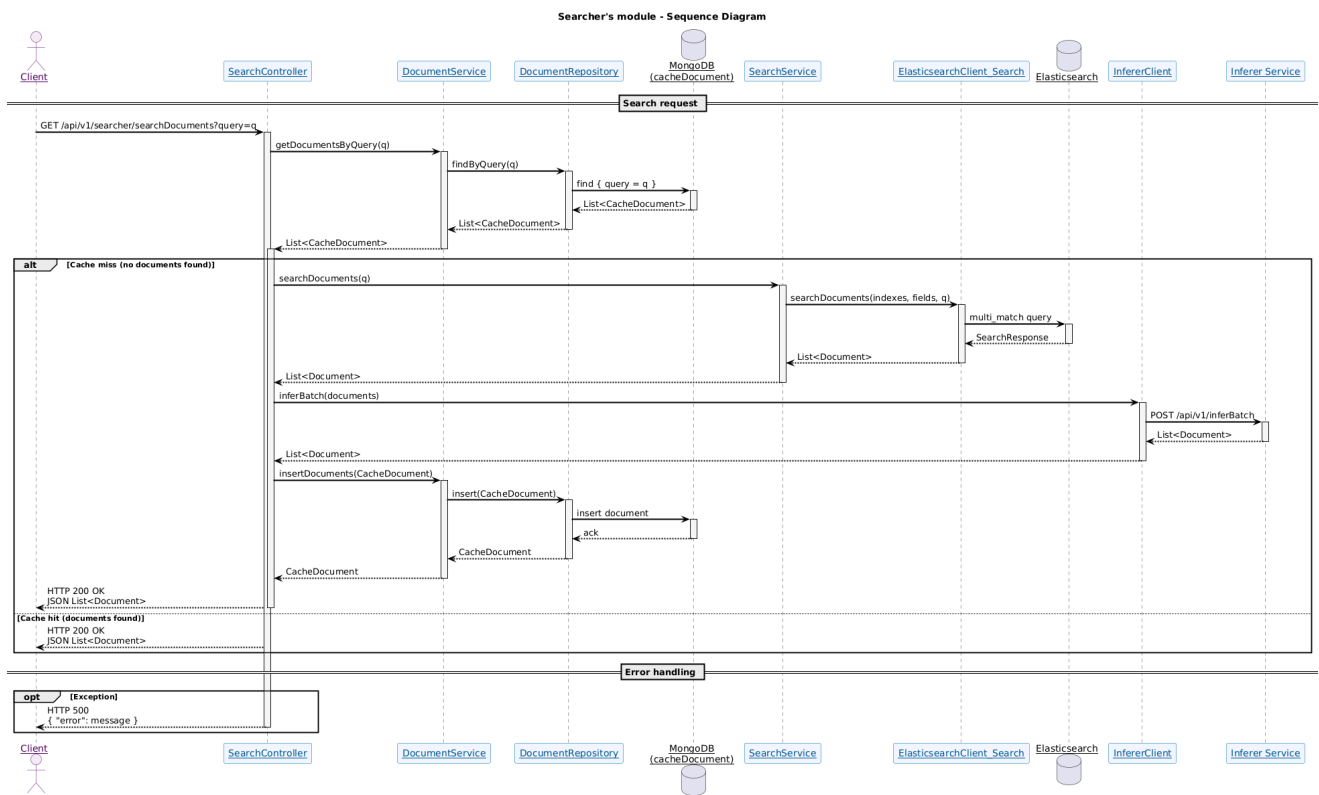
Activity Diagram - Searcher and Inferer System



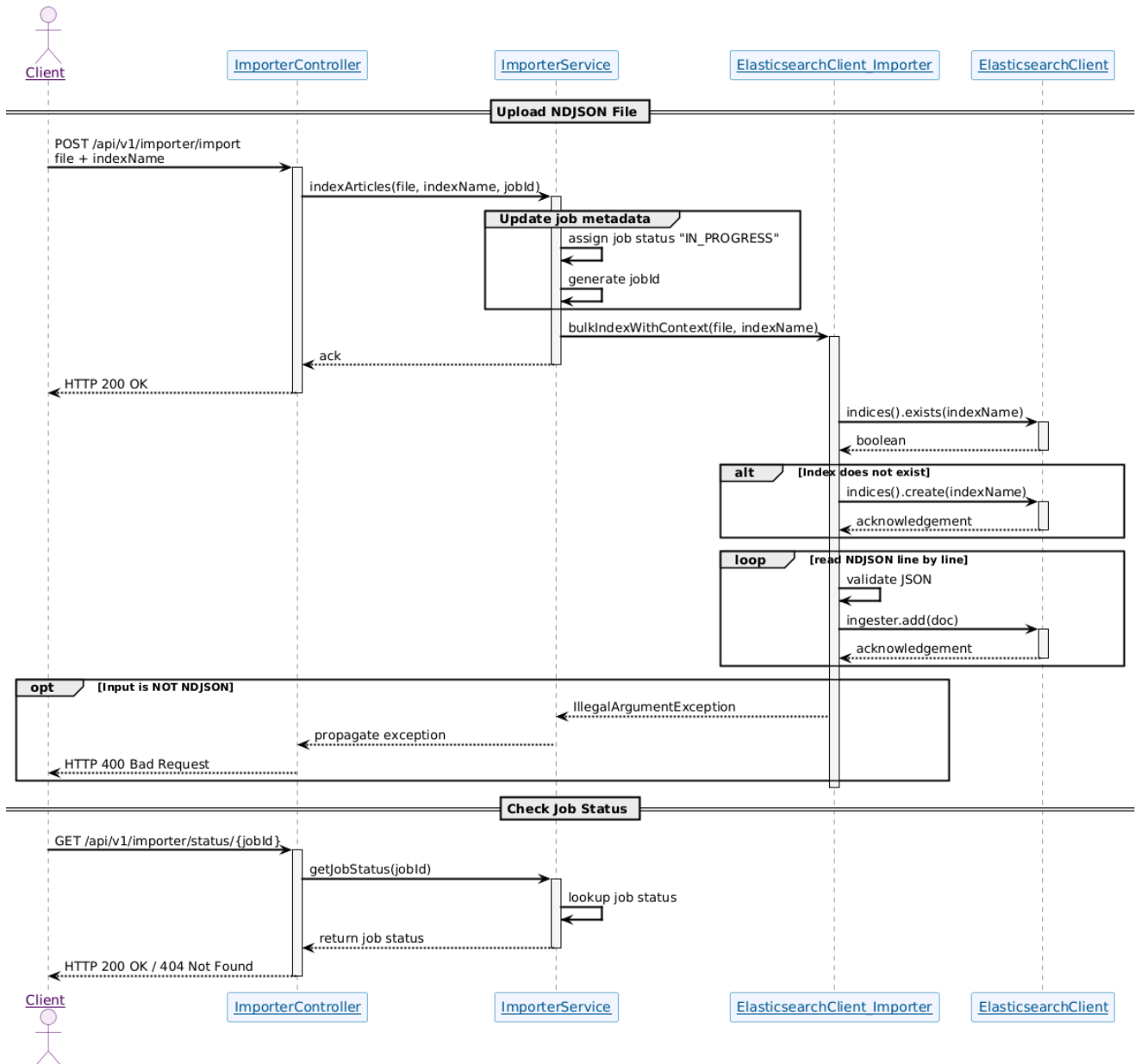
Activity Diagram - Importing System



Sequence diagrams



Importer module - Sequence diagram

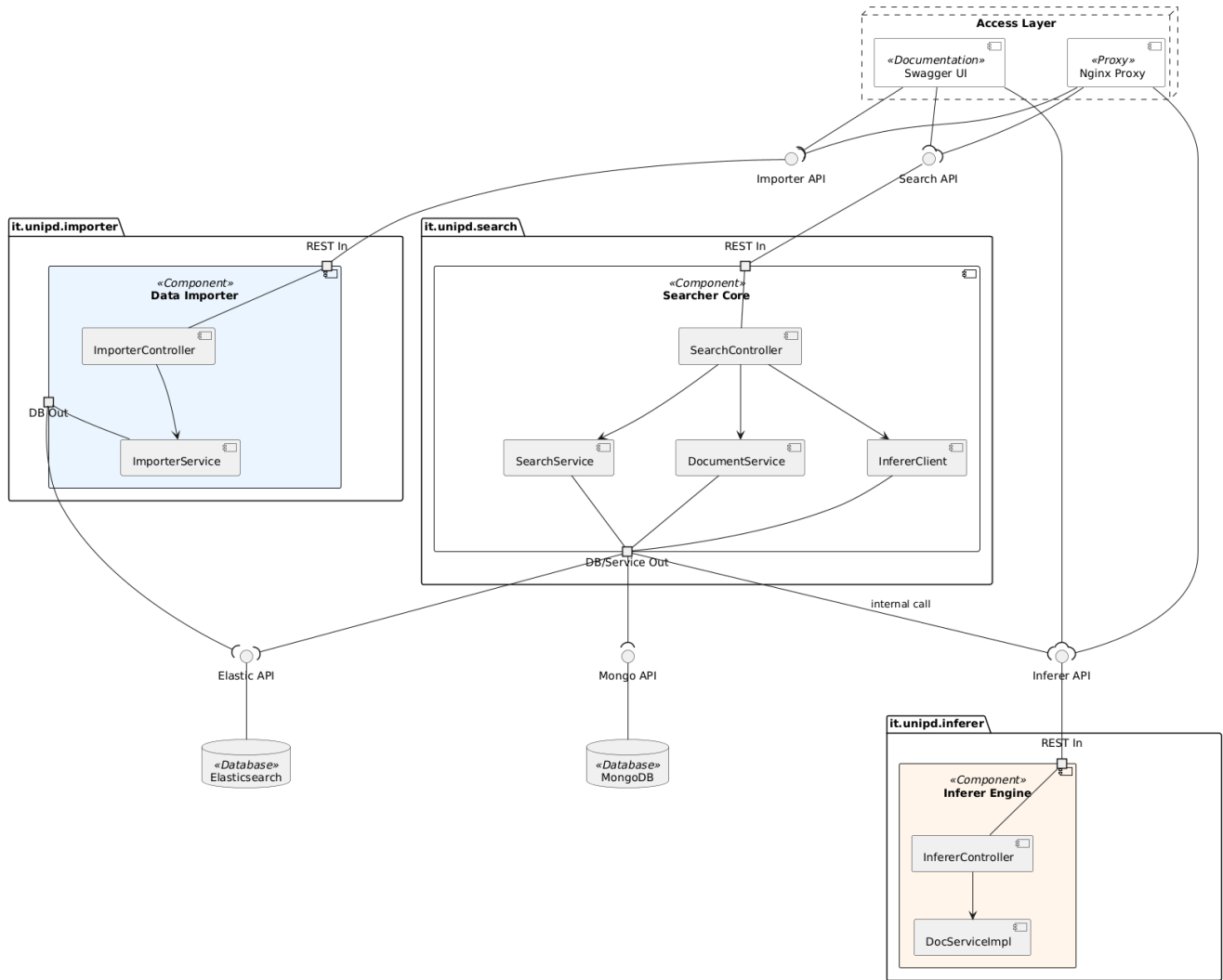


Implementation View

It guides **developers** and **project managers** in understanding code structure and facilitating team coordination during software management.

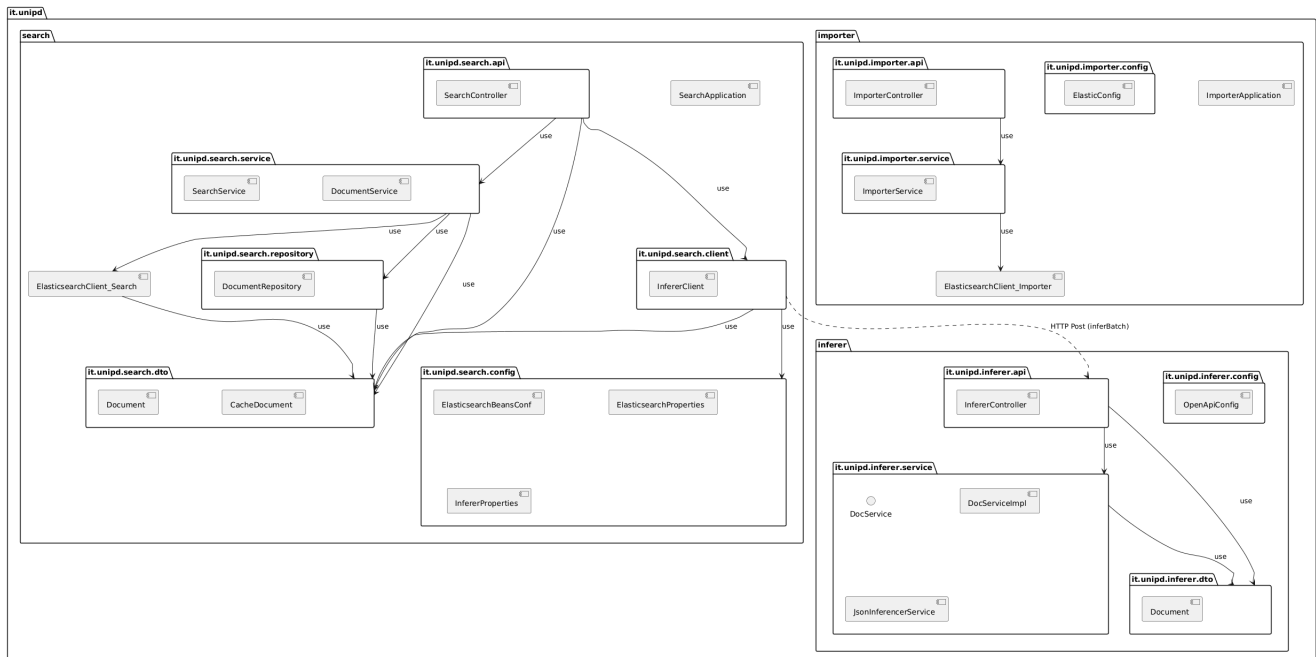
Component diagram

Searcher, importer and inferer's Component Diagram



Package diagram

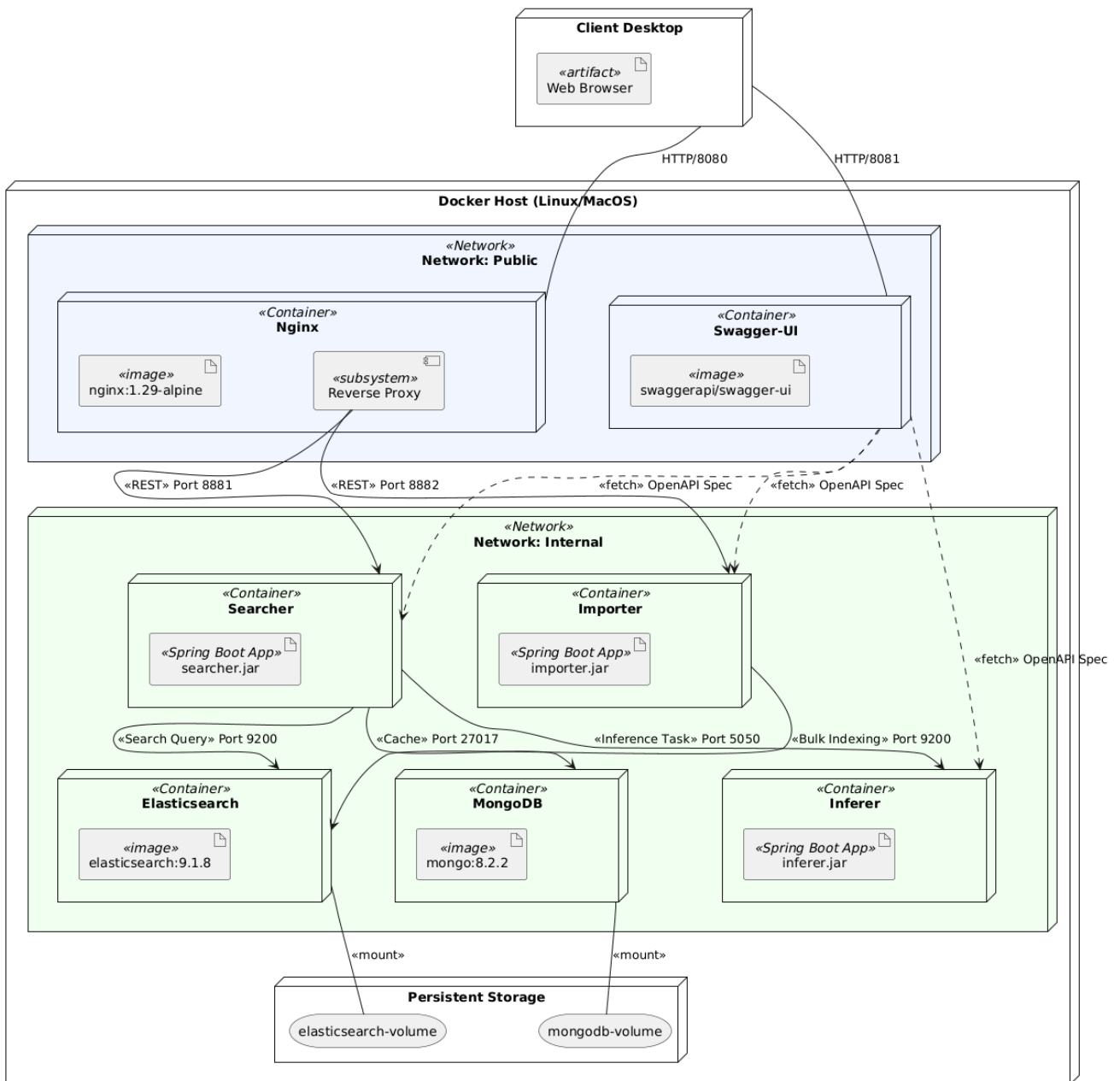
Package Diagram - Searcher, Importer & Inferer Modules



Deployment view

It shows to **software engineers Physical View** how software components are distributed across hardware infrastructure.

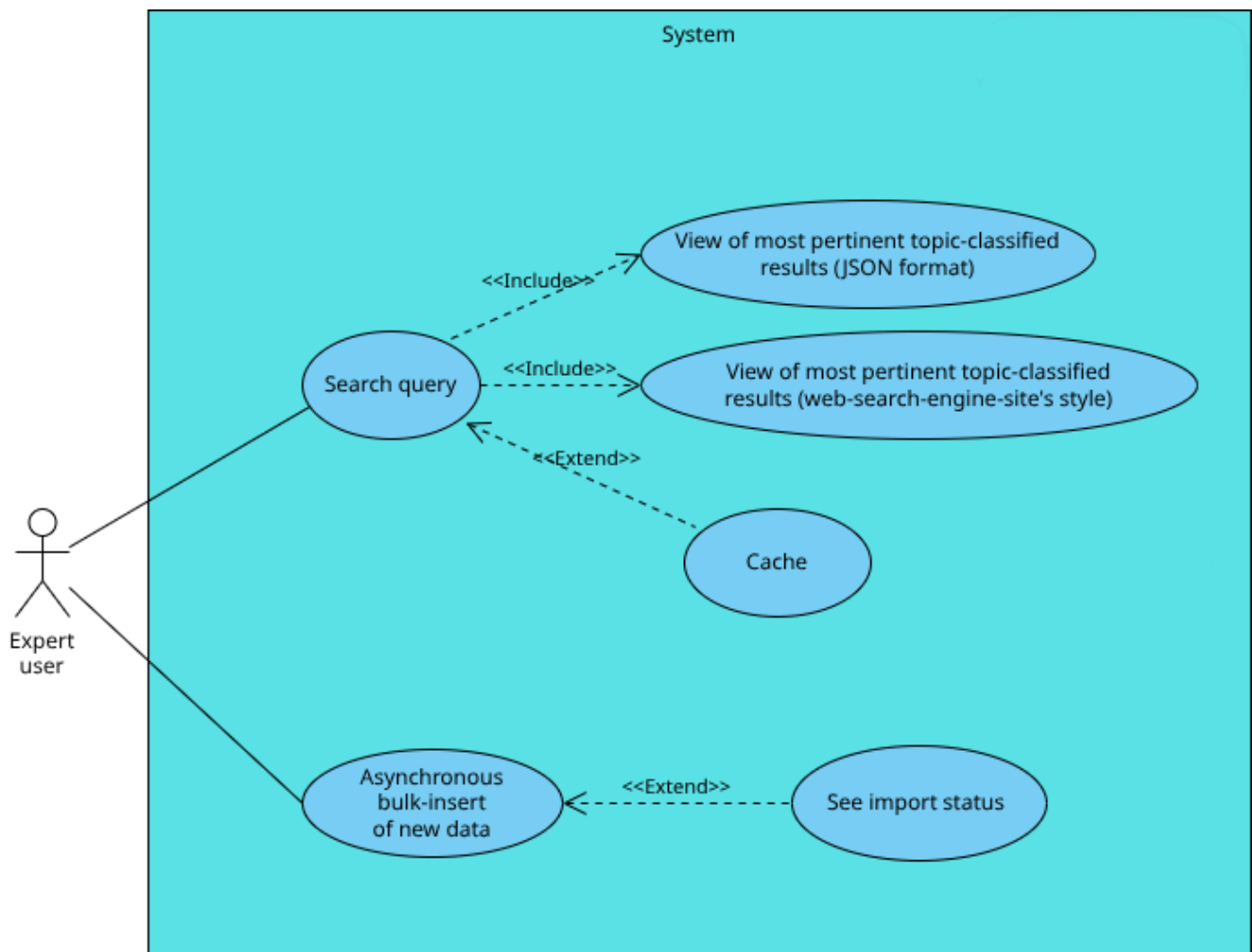
Deployment Diagram - Full project deployment



Use case View

It uses case diagrams to capture the system's functionality from the end-user perspective, serving as a unifying element that validates the other four views and ensures they collectively satisfy the system's requirements.

Use case diagram



User stories

The following user stories are written in standard Agile format: **As a ... I want ... so that**

They are directly aligned with the project's task and the professor's (many) tips given to us during the weeks of teaching.

User story 1 - Text document search

As an expert user,
I want to search documents using a free-text query,
so that I can quickly retrieve relevant documents from multiple online sources.

User story 2 – Searching multiple sources

As an expert user,
I want the system to search multiple document collections simultaneously,
so that I can obtain a broader and more comprehensive set of results.

User story 3 – Ranking based on relevance

As an expert user,
I want documents to be ranked with more importance given to titles than to full text,
so that the most relevant results appear first.

User story 4 – Retrieving results from the cache

As an expert user,
I want previous searches to return instant results (cache system),
so that I can improve my productivity during exploratory analysis.

User story 5 – Semantic enrichment of documents

As an expert user,
I want retrieved documents to be automatically enriched with inferred topics,
so that I can better understand their content and thematic relevance.

User story 6 – Configurability of external services

As a system administrator,
I want to configure the endpoints of external services via application properties,
so that the system can be deployed in different environments without code changes.

User story 7 – Exposure of a REST API

As a developer,
I want to access the search functionality via a REST API,
so that it can be easily integrated into other applications or user interfaces.

User story 8 – Error handling

As an expert user,
I want the system to return clear error responses when failures occur,
so that I can handle errors correctly.

User story 9 - Components are up and running

As a developer,
I want to verify that the search service is up and running
so that I can confirm the system is operational.

User story 10 - Not exact match

As an expert user,
I want to search for documents with a query that may not have an exact match
so that I can get relevant, approximate results.

User story 11 - Independent services

As a DevOps engineer,
I want to deploy the search service and inferer service independently
so that I can update or scale them separately.

Architecture Justification

Purpose and Context

The system is designed to support large-scale analysis of textual data (millions of documents) originating from heterogeneous sources. Its primary users are expert analysts who require expressive full-text search, reproducible analytical workflows and topic-based exploration with response times on the order of seconds. The system is deployed in a controlled, on-premises environment and prioritizes modularity and experimentation over consumer-grade latency.

Architectural Overview

The architecture follows a modular microservices pipeline:

1. Custom Importer ingests and normalizes source data.
2. Elasticsearch serves as the primary full-text search backend.
3. NGINX acts as a reverse proxy in front of the UI and APIs.

4. Query Handling Service retrieves cached results from MongoDB or executes new searches against Elasticsearch.
5. Topic Modeling Service (MALLET) performs analytical enrichment.
6. Query results and analytical outputs are cached for reuse.
7. Swagger generates documentation for each microservice's API.

This separation of concerns ensures scalability, maintainability, and analytical flexibility.

Roles and Justifications of Core Design Choices

Elasticsearch as Search Backend

Elasticsearch was chosen due to its ability to efficiently index and query millions of unstructured documents. Its inverted index, ranking capabilities, and aggregation support make it well-suited for exploratory full-text search. Given that query latency of a few seconds is acceptable, Elasticsearch provides an optimal balance between performance and expressiveness for expert users.

MongoDB for Query Caching and Analytical Results

MongoDB is used to store cached queries and derived analytical outputs. As a document-oriented database, it allows flexible schemas that accommodate evolving query definitions, topic modeling parameters, and metadata. This choice supports rapid experimentation and avoids rigid coupling between cached results and the underlying search index structure.

MALLET for Topic Modeling

MALLET was selected as the topic modeling engine due to its status as an academic standard. It provides well-understood, reproducible topic models suitable for research and expert analysis. Decoupling topic modeling from search allows analytical methods to evolve independently of retrieval infrastructure.

Custom Importer

A custom importer enables controlled preprocessing, normalization, and enrichment of heterogeneous data sources. In a multi-node cluster isolating write-heavy ingestion from read-heavy querying, the system maintains search performance while allowing re-ingestion or re-indexing without impacting downstream services.

Reverse Proxy and Controlled Exposure

NGINX provides a single, secure entry point to the system. Backend services are not directly exposed and API ports remain closed within the internal network. This design reduces attack surface in an on-premises deployment.

Swagger API generated docs

Swagger is used to generate the API docs for each one of the microservices which implements a controller. The Swagger microservice retrieves the information from each microservice and generates a webpage containing all the API docs. This provides a clear separation of concerns as well as a unique place where to consult the API docs. The API endpoints described in the documentation refer to the internal private network used for inter-service communication. While explicitly exposing internal network addresses in the documentation may appear to conflict with security best practices, this choice is intentional and limited to documentation purposes only. It is adopted to clearly describe and document the system architecture and service interactions and does not reflect the actual exposure of these endpoints in a production environment. In fact, once the user understands how the application works, removing Swagger from the running environment is very easy: the entire service can be commented out in Docker Compose, and the call within the Spring project is explicit and compartmentalised in a specific class in each module.

Custom trainer

The project includes the custom trainer used to train the MALLET topic modeling model.

This choice was made to ensure transparency and reproducibility of the analytical workflow. By providing the trainer as part

of the source code, expert users are able to retrain the model using their own datasets and adjust preprocessing steps or modeling parameters according to their specific domain needs. This flexibility is particularly important in research-oriented contexts, where topic quality and interpretability depend strongly on the underlying corpus.

Including the trainer also guarantees consistency between training and inference, as the same data processing logic is reused. Training is treated as an offline, resource-intensive operation and is therefore decoupled from the runtime query pipeline. New models can be generated and deployed without affecting system availability or query performance, supporting experimentation while preserving system stability.

Separated Owilix Dockerfile

The Dockerfile used to build the Owilix image is separated from the main project Dockerfile to preserve modularity and optionality in the data ingestion process. The use of OpenWebIndex data is not a mandatory requirement for the system, as alternative datasets, such as Guardian corpus, are already available and sufficient to populate the pipeline. Additionally, Owilix is not the only possible mechanism for acquiring OWI data; users may choose to directly download datasets from the Open Web Index website, filter and ingest them using the custom scripts and importer.

By isolating the Owilix Dockerfile, the architecture avoids introducing unnecessary dependencies for users who do not require OWI data or who prefer alternative data acquisition workflows. This separation simplifies deployment, reduces build complexity, and aligns with the system's experimentation-oriented design philosophy. Detailed instructions for using Owilix, when chosen, are provided in the project README.

This option ensures modularity and prevents a fixed data ingestion strategy.

Architectural Benefits

- **Scalability:** search and analysis workloads are isolated and can be scaled independently.
- **Modularity:** each component can be replaced or extended with minimal impact.
- **Experimentation:** flexible schemas and cached analytical results support iterative research workflows. Also alternatives are provided to customize the use.
- **Fault Isolation:** failures in non-essential services do not compromise ingestion or search.
- **Reproducibility:** cached queries and topic models enable consistent analytical results.

Deployment Considerations

All services are containerized using Docker to ensure reproducibility and ease of deployment. The system is deployed on-premises to maintain control over data and infrastructure. Resource-intensive components such as Elasticsearch and topic modeling services are isolated to prevent contention. Security is enforced through network segmentation, reverse proxy as only access to the internal network and minimal service exposure.

Conclusion

This architecture intentionally prioritizes analytical rigor, modularity, and controlled scalability. By combining proven search technology, flexible data storage, and academically established modeling tools, the system provides a robust foundation for expert-driven exploration of large textual corpora while remaining adaptable to future research and technical evolution.

Our project vs the competition

By comparing ourselves with our colleagues during the course of the project, we were able to analyse and understand their *modus operandi*. Our discussions revealed several points where our project differs from the competition. Here we will list the main ones.

- As far as we know, **we are the only group that seriously considered and managed to integrate OWI into the project**. The other groups' reasoning was that "it took too much time to get it working and the documentation was out of date". As good students **who believe in the ideals of open source and research**, we contacted a professor involved in the project (we will not name him for privacy reasons) and, by providing our logs and our collaboration, we have arrived at a service that, at the time of writing this document (06/01/2026), is fully functional. Specifically, the dataset we provided is a subset of a 35GB dataset filtered and exported to JSON. We decided to share this information with our colleagues in the hope that they too will choose to use this service, given the motivation and effort that researchers are putting into developing it.
- We know that code is only part of a good project. While many of our colleagues have limited themselves to javadoc documentation, we provide users with **not only all the javadoc documentation for the entire project, but also graphs and diagrams to help them understand our project intuitively and as fully as possible!** We have based our approach on the "4+1 view model", providing graphs for expert users and programmers as well as consumer users.
- The project brief explicitly stated that the application should be used by an expert user. Although an expert user is not defined (is it a programmer who is familiar with the terminal or someone who prefers to use graphical tools?), we tried to accommodate both. As stated in the documentation, searches on imported datasets can be performed either from a browser via a page very similar to Google's search page (used as inspiration to improve the usability and familiarity of the product) or from a terminal using, for example, the curl command. **Even users who are not familiar with the terminal can use our application. The README is copy-and-paste ready, and after installation and import, the application is ready to be used via the GUI.**
- The result is also accommodating for all types of expert users! From the terminal, you will get the results in JSON, while from the GUI you can get both pages filtered with Google-style topics and the JSON response that can be copied and pasted using the browser's 'Inspect' tool (tested on Chromium-based browsers; on Firefox-based browsers, the procedure is a little longer because you have to analyse the response packet, but it is fully functional, covering almost all types of browser technology).
- We've thought of everything! In our project, you'll find all the scripts you need in the *scripts* folder, all documented and ready to use, allowing you to install the application and get to work in less than 20 minutes^[1], whether you're a programmer or a regular user!
- As computer engineers, we know that we often find ourselves performing the same searches multiple times. **To offer our users a better experience**, we have introduced a caching system that allows for almost instantaneous response times after performing a search for the first time.
- Some of our colleagues mocked us for our simple, clean interface, showing us their polished and aesthetically appealing front ends. When we asked them how they had created those beautiful interfaces, they replied, 'We used Gemini and React.js.' After some discussion, we exposed the critical security flaws discovered this week in the library (more at this link <https://react.dev/blog/2025/12/03/critical-security-vulnerability-in-react-server-components>) and suggested caution until more was known, as the research team does not rule out other similar vulnerabilities. **We decided to keep things simple and functional, avoiding unnecessary frills and remaining faithful to well-established and stable patterns (reverse proxy with static pages)**. We hope that our colleagues have taken the necessary countermeasures to manage the security of users who will use their application. **We really hope so.**
- Regarding the previous point, we use Nginx, the most popular reverse proxy. **All incoming and outgoing traffic passes through this service, and there is no way to connect to internal services without going through it.** Our colleagues, on the other hand, decided to cut corners by leaving this aspect out, leaving doors open (e.g. binding in docker compose files) and thus (probably) allowing users to access every service, bypassing any encapsulation and compartmentalisation constraints.

Popularity of Nginx in April 2025

As of April 2025, W3Tech's web server count of all websites ranked Nginx first with 33.8%. Apache was second at 26.4% and Cloudflare Server third at 23.4%.

-[Wikipedia](#)

1. Time calculated using an 80Mb/s network and an average computer; times may vary depending on many factors. ↩